

CSV in JavaScript — Appunti

Cos'è un CSV?

CSV sta per "Comma Separated Values" (valori separati da virgola). È un modo semplicissimo per scrivere dati tabellari (come una tabella di Excel) dentro un normale file di testo: ogni riga di testo corrisponde a una riga della tabella, e i valori dentro la riga sono separati da virgole. La prima riga di solito è l'header, cioè contiene i nomi delle colonne.

```
const csvEsempio = `
nome, cognome, voto
Luca, Rossi, 7
Giulia, Bianchi, 9
Marco, Verdi, 5
`;
```

Step 1: trim()

Quando scriviamo un CSV su più righe con i backtick, spesso rimane una riga vuota all'inizio e/o alla fine. trim() rimuove gli spazi e i caratteri di "a capo" solo all'inizio e alla fine della stringa, lasciando intatto il contenuto centrale.

```
let csv = csvEsempio.trim();
```

Step 2: replaceAll(' ', '')

Nel CSV di esempio, dopo ogni virgola c'è uno spazio (" ") per renderlo più leggibile. Per lavorarci facilmente conviene toglierlo: replaceAll(' ', '') sostituisce OGNI spazio della stringa con una stringa vuota, cioè lo elimina.

```
csv = csv.replaceAll(' ', '');
// "nome,cognome,voto\nLuca,Rossi,7\n..."
```

Step 3: split('\n')

split('\n') taglia la stringa in corrispondenza di ogni "a capo" (newline) e restituisce un array: ogni elemento è una riga del CSV, ancora come stringa unica (non ancora divisa nelle singole colonne).

```
const rows = csv.split('\n');
// ["nome,cognome,voto", "Luca,Rossi,7", ...]
```

Step 4: shift()

shift() è un metodo degli array che RIMUOVE il primo elemento dell'array e lo RESTITUISCE come valore. Dopo questa riga "header" contiene la prima riga (i nomi delle colonne) e "rows" non la contiene più: shift() modifica l'array originale, non ne crea uno nuovo.

```
const header = rows.shift();
// header: "nome,cognome,voto"
// rows: ["Luca,Rossi,7", "Giulia,Bianchi,9", ...]
```

Step 5: split(',') sull'header

L'header per ora è ancora una stringa unica ("nome,cognome,voto"). Con split(',') la trasformiamo in un array con i nomi delle colonne.

```
const colonne = header.split(',');  
// ["nome", "cognome", "voto"]
```

Step 6: map() per costruire un array di dizionari

Vogliamo trasformare ogni riga di testo (es. "Luca,Rossi,7") in un dizionario tipo { nome: "Luca", cognome: "Rossi", voto: "7" }. Usiamo map() per creare un nuovo array (uno per ogni riga): dentro, dividiamo la riga con split(',') e costruiamo il dizionario usando "colonne" come chiavi.

```
const studenti = rows.map((row) => {  
  const valori = row.split(',');  
  const dizionario = {};  
  colonne.forEach((nomeColonna, indice) => {  
    dizionario[nomeColonna] = valori[indice];  
  });  
  return dizionario;  
});
```

Step 7: accedere ai dati del dizionario

Ogni elemento di "studenti" è un dizionario: possiamo leggerne i campi con la notazione a punto, esattamente come un oggetto qualsiasi.

```
studenti.forEach((studente) => {  
  console.log(`${studente.nome} ha preso ${studente.voto}`);  
});
```

Step 8: generare un CSV risultato con map() + join('\n')

Percorso inverso: da un array di dizionari a una stringa CSV. Prima creiamo la riga di header unendo i nomi delle colonne con join(','), poi per ogni studente costruiamo la riga dei valori con Object.values() + join(','), infine uniamo tutte le righe con join('\n').

```
const headerRisultato = colonne.join(',');  
const righeRisultato = studenti.map((s) => Object.values(s).join(','));  
const csvRisultato = headerRisultato + '\n' + righeRisultato.join('\n');
```

Step 9: template.replace() come alternativa

Invece di costruire ogni riga con Object.values().join(','), possiamo scrivere un template con dei segnaposto (es. "{{nome}}") e sostituirli con replace(). Utile quando l'ordine o il formato del risultato è diverso da quello del CSV di partenza, o non è nemmeno un vero CSV.

```
const templateRiga = "{{nome}} {{cognome}}: {{voto}}/10";  
  
const righeConTemplate = studenti.map((s) => {  
  templateRiga  
    .replace('{{nome}}', s.nome)  
    .replace('{{cognome}}', s.cognome)  
    .replace('{{voto}}', s.voto)  
});
```